PART B — PROJECT MANAGEMENT & SPRINT TRACKING [50 Points]

SECTION 4 — AGILE / SCRUM FUNDAMENTALS [12 pts]

Task 4.1 — Agile Knowledge Questions

Q1 — Product Backlog vs Sprint Backlog vs Increment

Artefact	Definition	E-Commerce Example
Product Backlog	Product Backlog is basically the main list of everything we need to build or fix in the product. It includes new features, improvements, and bugs. This list is handled by the Product Owner, and it keeps changing based on customer needs, business priorities, or feedback from production issues	In an e-commerce project I worked on, the Product Backlog had items like: Adding a wish-list feature , Improving search filters for out stocks products Fixing payment failure re-directly to correctly or retry machnisum or refund correctly to wallets and bank accounts Adding order tracking notifications If a production issue comes, like payment failing for UPI users, that bug is immediately added.
Sprint Backlog	Sprint Backlog is the set of items picked from the Product Backlog for the current sprint, mainly based on priority and business importance. In real scenarios, we don't pick everything—we only take the items that the team feels confident they can complete within that sprint	For example, in an e-commerce app, suppose the business wants to improve user experience during a sale. So for that sprint, instead of adding completely new features, the team might pick things like: Improving product search speed Fixing filters not working properly for some categories Making the 'Add to Cart' button more responsive Handling edge cases where items go out of stock during checkout
Increment	Increment means whatever we planned in the sprint is now actually working in the application. It's not just coding done—it includes testing, bug fixing, and making sure everything is stable.	suppose we worked on product reviews. At the end of the sprint: User can add a review Rating is saved Reviews are displayed properly That complete working feature becomes the Increment

Q2 — The 5 Scrum Events

Scrum Event	Purpose	Time-Box
Sprint	A sprint is like a short cycle where we plan some work, develop it, test it, and make sure it's fully working. By the end of the sprint, whatever we worked on should be in a 'done' state and ready to use not partial or incomplete.	1–4 weeks (this project: 2 weeks)
Sprint Planning	In sprint planning, the team first decides a clear goal for the sprint, and then selects the relevant items from the backlog and plans how to complete them within that sprint.	Max 8 hours for 1-month sprint; ~4 hrs for 2-week sprint
Daily Scrum	We have a daily standup for around 15 minutes. Everyone shares what they worked on yesterday, what they are planning today, and if there are any blockers. Based on that, we adjust our work to make sure we are on track to achieve the sprint goal	15 minutes (same time, same place)

Sprint Review	At the end of the sprint, we show the completed work to stakeholders, get their feedback, and update the backlog if any changes or new requirements come	Max 4 hours for 1-month sprint; ~2 hrs for 2-week sprint
Sprint Retrospective	At the end of the sprint, the team sits together and discusses what went well, what didn't go well, and how we can improve in the next sprint	Max 3 hours for 1-month sprint; ~1.5 hrs for 2-week sprint

Q3 — Release Date Estimation
Based on our team capacity, we estimate around 16 weeks of work, so starting June, the release will be around end of September considering some buffer.

Task 4.2 — RACI Matrix

Activity	Product Owner	Scrum Master	Dev Lead	QA Lead	Stakeholder
Sprint Planning	A	R	R	R	I
Daily Standup	I	R	R	R	I
Backlog Refinement	A	R	C	C	I
Sprint Review	A	R	R	R	C
Sprint Retrospective	I	A/R	R	R	I
Defect Triage	C	C	R	A	I
Release Sign-off	A	C	R	A	I

R = Responsible | A = Accountable | C = Consulted | I = Informed

SECTION 5 — SPRINT PLANNING & BACKLOG MANAGEMENT [14 pts]

US-001 — User Registration

Description:
As a new user, I want to register using my email and password so that I can access the application and place orders.

Acceptance Criteria:

- User should be able to register with valid email & password
- System should not allow duplicate email registration
- Password should follow rules (min 8 chars, number, etc.)
- Proper error message should be shown for invalid inputs
- On successful registration, user account should be created in DB
- User should be able to login after registration

Story Points: 5
Priority: Must Have

Dependencies:

- User service API
- Database

QA Approach (real working style):

- Validate UI + API + DB entry
- Negative testing (invalid email, blank fields, duplicate user)
- Check response codes (200 / 400)
- Basic security checks (password masking)

US-002 — Product Search & Filter

Description:
As a user, I want to search and filter products so that I can quickly find relevant items.

Acceptance Criteria:

- Search should return products based on keyword
- Filters like category, price should work correctly
- Multiple filters should work together
- Clearing filters should reset results
- No results → proper message shown
- Pagination / scroll should load correct data

Story Points: 8
Priority: Must Have

Dependencies:

- Product API
- Search service

QA Approach:

- Validate combinations of filters
- Edge cases (empty search, special characters)
- Data validation between UI and API
- Performance check (basic — not strict in early sprint)

US-003 — Add to Cart

Description:
As a user, I want to add products to my cart so that I can purchase them later.

Acceptance Criteria:

- User should be able to add product to cart
- Cart count should update immediately
- Adding same product → quantity increases
- User should be able to remove items
- Cart total should update correctly
- Cart should persist for logged-in users

Story Points: 5
Priority: Must Have

Dependencies:

- Cart API
- User session

QA Approach:

- Verify UI vs API cart data
- Refresh / logout scenarios
- Multiple product handling
- Price calculation validation

US-004 — Promo Code

Description:
As a user, I want to apply promo code so that I can get discount on my order.

Acceptance Criteria:

- Valid promo code should apply discount
- Invalid/expired code should show error
- Only one promo code allowed
- Discount should reflect in total price
- Promo should not apply if conditions not met (min cart value)

Story Points: 3
Priority: Should Have

Dependencies:

- Pricing service

QA Approach:

- Boundary testing (expiry, min value)
- Verify discount calculation
- Multiple negative scenarios

US-005 — Checkout & Payment

Description:
As a user, I want to complete payment so that I can place an order successfully.

Acceptance Criteria:

- User should be able to enter shipping details
- Multiple payment options available
- Payment success → order created
- Payment failure → proper error shown
- User should not be charged multiple times on retry
- Order confirmation page should be displayed

Story Points: 13
Priority: Must Have

Dependencies:

- Payment Gateway
- Order service

QA Approach:

- Test success & failure flows
- Retry scenarios
- Verify order creation after payment
- Validate payment response

US-006 — Order History

Description:
As a user, I want to view my past orders so that I can track them.

Acceptance Criteria:

- Orders displayed latest first
- Each order should show details (items, price, status)
- User should be able to open order details
- Basic filter should work
- Order data should match actual placed order

Story Points: 3
Priority: Should Have

QA Approach:

- Data validation
- UI vs backend verification
- Status accuracy

Sprint Backlog (Capacity: 40 Story Points)

Story ID	Story Title	Story Points	Assignee	Status

US-001	User Registration	5	Dev1	To Do
US-002	Product Search & Filter	8	Dev2	To Do
US-003	Add to Cart	5	Dev3	To Do
US-004	Promo Code Application	3	Dev2	To Do
US-006	Order History	3	Dev1	To Do
US-005	Checkout & Payment	13	Team	Stretch

Total: 37 Story Points
Note: Remaining buffer (~3 SP) is kept for bug fixing and rework.

Task Breakdown (Jira-style Subtasks)

US-001 — User Registration

Development Tasks:

- Create registration UI
- Implement input validation
- Develop registration API
- Integrate with database

QA Tasks:

- Write test cases
- Execute test cases
- Log defects
- Retest after fixes

US-003 — Add to Cart

Development Tasks:

- Develop cart UI
- Implement add/remove cart API
- Handle quantity update logic
- Maintain session/cart persistence

QA Tasks:

- Functional testing of cart
- Edge case testing (multiple items, refresh)
- Regression testing

Day-to-Day Execution (During Sprint)

- Daily standup to discuss progress, plan, and blockers
- Developers work on assigned tasks and move status in Jira
- QA starts testing once build is available
- Bugs are logged and tracked in Jira
- Developers fix issues and QA retests
- Continuous updates in Jira (To Do → In Progress → Testing → Done)

Definition of Done (DoD)

A user story is marked as Done only when:

- Development is completed
- Code is reviewed and merged
- QA testing is completed
- No critical or high severity bugs are open
- Feature is deployed in QA environment
- All acceptance criteria are met

Risks / Blockers

- Delay in backend APIs → handled using mock data
- Payment gateway dependency for checkout
- Requirement clarification delays
- Environment or build issues
- Integration issues between modules

SECTION 6 — SPRINT TRACKING & PROGRESS REPORTING [14 pts]

Task 6.1 — Sprint Burndown Chart & Analysis

Day	1	2	3	4	5	6	7	8	9	10
Ideal SP Remaining	40	36	32	28	24	20	16	12	8	4
Actual SP Remaining	40	38	35	34	30	25	22	18	12	6

[Burndown Chart — ASCII Representation]

```
SP  |
40  | *  A
36  |  * . A
32  |    *   A
28  |     *    A
24  |      *      A
20  |       *        A
16  |        *          A
12  |         *            A
```

```
8  |          *              A
4  |           *                A
0  +--+--+--+--+--+--+--+--+--+--
    D1 D2 D3 D4 D5 D6 D7 D8 D9 D10
   * = Ideal Line  |  A = Actual Line
```

Burndown Analysis:

- Overall Trajectory: The team is slightly behind the ideal burndown line throughout the sprint, but the gap is closing — from 4 points behind on Day 2 to only 2 points behind by Day 10. The sprint is recoverable.
- Early Sprint (Days 1–4): The team burned fewer points than the ideal rate, suggesting slow ramp-up, possibly due to dependency resolution (e.g., environment setup, SMTP configuration) or time spent in refinement.
- Mid-Sprint Recovery (Days 5–7): Burn rate accelerates and begins converging with the ideal line — suggests blockers were resolved and development momentum improved.
- End of Sprint (Days 8–10): Actual (6 pts remaining) vs Ideal (4 pts remaining) — a 2-point gap, indicating ~95% completion. The sprint goal is largely met.
- Recommended Actions: Investigate Days 1–3 slowdown in retrospective. Ensure early blockers (environment, credentials) are resolved by Sprint Day 1 in Sprint 2. The team should aim to close the 2-point gap via a 30-minute scope negotiation with the PO.

Task 6.2 — Sprint Status Report (End of Day 7)

Section	Detail
Sprint Goal	Users can register, log in, browse products, and add items to cart — foundational e-commerce flow delivered.
Sprint Dates	01 June 2025 – 14 June 2025 (Day 7 of 10: 07 June 2025)
Team Members	Priya (Dev), Arjun (Dev), Sana (Dev), Ravi (QA), Maya (Scrum Master), Vikram (Product Owner)
SP Burned (Day 7)	40 − 22 = 18 story points burned | 22 story points remaining
Ideal Burn by Day 7	40 − 16 = 24 story points burned | 16 ideal remaining
Variance	−6 SP behind ideal on Day 7 (recovering — gap was −8 on Day 4)
Stories Completed	US-001 (User Registration, 5 pts) — Done
Stories In Progress	US-002 (Product Search, 8 pts) — 70% complete | US-003 (Add to Cart, 5 pts) — 60% complete
Stories Not Started	US-004 (Promo Code, 3 pts) | US-006 (Order History, 3 pts)
Key Achievements	User registration and login fully functional and tested. Email verification workflow live in staging. Product listing page rendering correctly across Chrome, Firefox, and Safari.
Current Blockers	SMTP email service intermittently failing in staging (Owner: DevOps — Anil; Resolution: migrate to Mailtrap by Day 8). Product filter API returning 500 errors on combined filter queries (Owner: Dev Arjun; ETA: Day 8 fix).
Risk Update	Checkout story (US-005, 13 pts) remains deferred to Sprint 2 — confirmed with PO on Day 5. New risk: QA bandwidth may constrain full regression if remaining 22 pts close on Days 8–9 simultaneously.
Next Steps (Days 8–10)	1. Resolve filter API bug (Arjun, Day 8). 2. Complete Cart feature (Sana, Day 8–9). 3. QA execution on US-002, US-003, US-004, US-006. 4. Sprint Review preparation (Vikram). 5. Retrospective agenda drafted by Maya.

SECTION 7 — STAKEHOLDER COMMUNICATION & SCENARIO QUESTIONS [10 pts]

Task 7.1 — Stakeholder Update Email

> From: [Your Name] | Scrum Master / Project Manager
> To: [Client Name] | Subject: Sprint 1 Status Update — E-Commerce Platform

Dear [Client Name],

I hope this message finds you well. I am writing to share a progress update at the close of Sprint 1 for the E-Commerce Platform project.

SPRINT 1 SUMMARY

Our team delivered 28 of the planned 40 story points this sprint. The foundational features — user registration, login, and product browsing — are complete, tested, and deployed to the staging environment. These provide a solid, working base for the product and represent strong forward momentum.

TWO STORIES WERE IMPACTED BY A THIRD-PARTY DEPENDENCY

The Checkout & Payment and Promo Code Application stories were blocked due to a delay in receiving sandbox API credentials from our payment gateway provider. This is a vendor-side delay outside our team's control. We have escalated directly with the provider and expect credentials by [Date + 3 days]. Both stories have been reprioritised to the top of the Sprint 2 backlog and will be completed within the next sprint cycle.

SPRINT 2 PLAN

Sprint 2 will deliver: Checkout & Payment (3 payment methods), Promo Code Application, Order History, and completion of Product Search & Filter. The team is at full capacity and well-positioned to deliver the complete shopping flow by end of Sprint 2.

I would welcome a 30-minute call to walk through the staging build live if you are available this week. Your feedback at this stage is invaluable to ensure we are aligned before the checkout features are built.

Thank you for your continued partnership. Please do not hesitate to reach out with any questions.

Warm regards,
Megha L m
Project Manager / Scrum Master
meghalm2000@gmail.com | 8553941159

Task 7.2 — Scenario-Based PM Questions

Q1 — Mid-Sprint Priority Change Request

- Acknowledge and Record: Thank the PO for raising the priority. Log the request formally in the backlog.
- Do Not Disrupt the Sprint: Protect the current sprint. Inform the PO that modifying the sprint backlog mid-sprint undermines the team's commitment and predictability.
- Impact Assessment: With the Dev Lead and QA Lead, quickly assess: What is the effort? Does it displace an existing story? What is the risk of introducing it now?
- Backlog Prioritisation: If genuinely urgent, collaborate with the PO to either: (a) Swap it with an equal or lesser story with team consent, or (b) Fast-track it to the top of Sprint 2 with an aggressive slot at the start of Sprint Planning.
- Escalation Path: If the change is business-critical (e.g., regulatory compliance, site-down risk), call an emergency Scrum event. Transparently communicate trade-offs to all stakeholders.
- Outcome: Document the decision (sprint change log) and communicate the result to the requester. Never silently absorb scope without the team's agreement.

Q2 — Developer Conflict Over Technical Approach

- Acknowledge the Conflict Early: Engage within 24 hours — unresolved technical conflicts compound.
- Understand Both Positions: Hold a brief, private 1:1 with each developer to understand their rationale without judgment. Separate the person from the position.
- Facilitate a Joint Technical Discussion: Bring both developers together in a structured 60-minute session. Ask each to present their approach against agreed criteria: performance, maintainability, team skill set, timeline, and risk.
- Involve a Neutral Expert: If consensus is not reached, invite the Tech Lead or an external architect for an objective recommendation.
- Make a Decision and Commit: Document the chosen approach and rationale. Both developers commit to the decision — even if one disagreed.
- Address the Delay Transparently: Update the sprint burndown. Inform stakeholders if timeline is affected (do not hide the delay). Review the root cause in the retrospective to prevent recurrence.
- Follow Up: Check in with both developers post-resolution to ensure no residual friction affecting team dynamics.

Q3 — Communicating QA Progress to Different Audiences

Audience	Format	Frequency	Key Metrics Included	Communication Channel
Development Team	Daily QA Dashboard / JIRA Board	Daily (updated each morning)	Test execution status per story, open defects by severity, blocked test cases, defect assignment owner	JIRA filters, Slack #qa-status channel, daily standup
QA Team Internal	Detailed TestRail / Test Management Report	Daily	TC pass/fail/blocked counts, coverage %, regression suite health, automation pass rate, defect ageing	TestRail, internal QA stand-up
Senior Management / Stakeholders	1-page Sprint QA Summary Report	End of sprint (or weekly for long sprints)	Overall quality health (green/amber/red), % stories tested and signed off, total defects raised vs closed, critical open risks, release readiness recommendation	Email, Sprint Review slide, PDF report

Key Principle: For engineers, give raw data and actionable detail. For management, give a traffic-light summary with business risk framing — avoid jargon. Always lead with 'Are we on track to release?' and support with metrics.

SCORING RUBRIC — SELF-ASSESSMENT

Part	Section	Topic	Max Pts	Coverage in This Submission
A	1	Test Planning & Analysis	15	Full test plan, 8 testing types, entry/exit criteria, 5-risk register, environment spec, Three-Point estimation
A	2	Test Case Design (BVA/EP)	15	15 functional test cases (all 8 scenarios), full BVA & EP table with 19 cases across 3 fields
A	3	Bug Reporting & Automation	20	4 formal bug reports (BUG-001 to BUG-004), full POM Selenium Python automation suite, bonus CI YAML

B	4	Agile / Scrum Fundamentals	12	All 3 questions answered, RACI matrix for 7 activities × 5 roles
B	5	Sprint Planning & Backlog	14	6 full user stories (Fibonacci, MoSCoW, 4 ACs each), Sprint Plan with goal, backlog, task breakdown, DoD
B	6	Sprint Tracking & Reporting	14	Burndown table + ASCII chart + 5-sentence analysis, full Day-7 Sprint Status Report
B	7	Stakeholder Communication	10	Professional stakeholder email, 3 scenario answers
—	—	TOTAL	100	All 100 points addressed — see individual sections for detail